

CSCE 46103/56103 Artificial Intelligence – Fall 2026

Bellman Equations, Value Iteration, and Policy Iteration

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

October 6, 2026

Today's Topics

- ▶ Value functions and Bellman equations.
- ▶ Value iteration and the propagation of future rewards.
- ▶ Policy evaluation, policy improvement, and policy iteration.
- ▶ Convergence, stopping criteria, and algorithm comparisons.

The Stochastic Robot Grid

3				+1
2		wall		-1
1	start			
	1	2	3	4

- ▶ State: the observed cell (x, y) .
- ▶ Actions: up, right, down, and left.
- ▶ Intended direction: probability 0.8.
- ▶ Each perpendicular slip: probability 0.1.
- ▶ A blocked move leaves the robot in place.

The Planning Problem

The transition and reward models are known. Choose an action for every nonterminal state.

Rewards and Terminal Values

For a nonterminal state s , use the following reward convention:

$$R(s, a, s') = \begin{cases} +1, & s' = (4, 3), \\ -1, & s' = (4, 2), \\ -0.04, & s' \text{ is nonterminal.} \end{cases}$$

- ▶ Terminal entry receives its listed reward alone.
- ▶ Every other transition costs 0.04, including a blocked move.
- ▶ After terminal entry, stay in that state and receive zero forever.

Rewards and Terminal Values

For a nonterminal state s , use the following reward convention:

$$R(s, a, s') = \begin{cases} +1, & s' = (4, 3), \\ -1, & s' = (4, 2), \\ -0.04, & s' \text{ is nonterminal.} \end{cases}$$

- ▶ Terminal entry receives its listed reward alone.
- ▶ Every other transition costs 0.04, including a blocked move.
- ▶ After terminal entry, stay in that state and receive zero forever.

Continuation After Arrival

Both terminal states have value 0. Their $+1$ and -1 labels denote entry rewards, not future returns after arrival.

Discounted Return and State Value

The return from time t is

$$G_t = \sum_{j=0}^{\infty} \gamma^j R(S_{t+j}, A_{t+j}, S_{t+j+1}).$$

For a deterministic stationary policy π , define

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

- ▶ It assumes finite state and action sets, bounded rewards, and $0 \leq \gamma < 1$.
- ▶ $V^{\pi}(s)$ is the expected return when starting at s and following π .
- ▶ Our grid uses $\gamma = 0.95$ and an infinite horizon.

The Recursive Return

Separate the next reward from the rest of the sequence:

$$\begin{aligned} G_t &= R(S_t, A_t, S_{t+1}) + \gamma R(S_{t+1}, A_{t+1}, S_{t+2}) + \cdots \\ &= R(S_t, A_t, S_{t+1}) + \gamma G_{t+1}. \end{aligned}$$

The Recursive Return

Separate the next reward from the rest of the sequence:

$$\begin{aligned} G_t &= R(S_t, A_t, S_{t+1}) + \gamma R(S_{t+1}, A_{t+1}, S_{t+2}) + \cdots \\ &= R(S_t, A_t, S_{t+1}) + \gamma G_{t+1}. \end{aligned}$$

- ▶ Once the next state is reached, the remaining decision problem has the same form.
- ▶ Its expected return is the value of that next state.

Bellman Policy Evaluation Equation

Fix a policy π . Its action at state s is $\pi(s)$:

$$V^\pi(s) = \sum_{s'} T(s' \mid s, \pi(s)) [R(s, \pi(s), s') + \gamma V^\pi(s')] .$$

1. Follow the action selected by the policy.
2. For each possible next state, add its immediate reward and discounted continuation value.
3. Average using the transition probabilities.

Bellman Policy Evaluation Equation

Fix a policy π . Its action at state s is $\pi(s)$:

$$V^\pi(s) = \sum_{s'} T(s' \mid s, \pi(s)) [R(s, \pi(s), s') + \gamma V^\pi(s')] .$$

1. Follow the action selected by the policy.
2. For each possible next state, add its immediate reward and discounted continuation value.
3. Average using the transition probabilities.

A System of Equations

There is one equation per state. Values depend on other values, and self-transitions can make a value depend on itself.

Action Values Under a Policy

Take action a now, then follow π :

$$Q^\pi(s, a) = \sum_{s'} T(s' \mid s, a) [R(s, a, s') + \gamma V^\pi(s')] .$$

State Value

$V^\pi(s)$ includes the action prescribed by π at the current state.

Action Value

$Q^\pi(s, a)$ allows a different first action. Future actions still follow π .

$$V^\pi(s) = Q^\pi(s, \pi(s)).$$

One-Step Lookahead with Estimated Values

For any candidate value table V , define

$$Q_V(s, a) = \sum_{s'} T(s' \mid s, a) [R(s, a, s') + \gamma V(s')].$$

- ▶ The transition and reward models remain fixed.
- ▶ Only the continuation table V changes during computation.
- ▶ When $V = V^\pi$, this gives $Q_V = Q^\pi$.

A Shared Calculation

Value iteration and policy iteration both compare these one-step lookahead values.

Optimal Values and the Bellman Equation

The optimal value is the best expected return available from each state:

$$V^*(s) = \max_{\pi} V^{\pi}(s).$$

Choose the best first action and continue optimally afterward:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} T(s' | s, a) [R(s, a, s') + \gamma V^*(s')] .$$

Optimal Values and the Bellman Equation

The optimal value is the best expected return available from each state:

$$V^*(s) = \max_{\pi} V^{\pi}(s).$$

Choose the best first action and continue optimally afterward:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} T(s' | s, a) [R(s, a, s') + \gamma V^*(s')] .$$

A Recursive Optimality Condition

The unknown values appear on both sides. The maximum couples action selection with the values of future states.

Expectations and Action Selection

$$V^*(s) = \underbrace{\max_a}_{\text{choose an action}} \underbrace{\sum_{s'} T(s' | s, a) [R(s, a, s') + \gamma V^*(s')]}_{\text{average its possible outcomes}} .$$

- ▶ The agent controls the intended action.
- ▶ The environment determines the random movement outcome.
- ▶ Each action needs its own expectation before actions can be compared.

A Slip Remains Possible

Choosing the best action does not allow the robot to choose which outcome occurs.

An Optimal Policy from Optimal Values

Once V^* is known, choose

$$\begin{aligned}\pi^*(s) &\in \arg \max_{a \in \mathcal{A}(s)} Q_{V^*}(s, a) \\ &= \arg \max_{a \in \mathcal{A}(s)} \sum_{s'} T(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')] .\end{aligned}$$

- ▶ $\max$ gives the largest action value; $\arg \max$ identifies actions attaining it.
- ▶ More than one action can be optimal at a state.
- ▶ A deterministic stationary optimal policy exists under our assumptions.

Greedy with Respect to a Value Table

The comparison includes expected immediate reward and discounted future value. A policy greedy with respect to an approximate table need not be optimal.

A Backup Beside the Positive Terminal

At $s = (3, 3)$, consider action right. Its outcomes are

Outcome	s'	$T(s' \mid s, \text{right})$	$R(s, \text{right}, s')$
Right	$(4, 3)$	0.8	+1
Up, blocked	$(3, 3)$	0.1	-0.04
Down	$(3, 2)$	0.1	-0.04

A Backup Beside the Positive Terminal

At $s = (3, 3)$, consider action right. Its outcomes are

Outcome	s'	$T(s' \mid s, \text{right})$	$R(s, \text{right}, s')$
Right	$(4, 3)$	0.8	+1
Up, blocked	$(3, 3)$	0.1	-0.04
Down	$(3, 2)$	0.1	-0.04

$$Q_V((3, 3), \text{right}) = 0.8[1 + 0.95(0)] + 0.1[-0.04 + 0.95V(3, 3)] + 0.1[-0.04 + 0.95V(3, 2)].$$

Self-Transitions in an Equation and an Update

If $\pi(3, 3) = \text{right}$, the fixed-policy equation is

$$V^\pi(3, 3) = 0.792 + 0.095V^\pi(3, 3) + 0.095V^\pi(3, 2).$$

Rearranging gives

$$0.905V^\pi(3, 3) = 0.792 + 0.095V^\pi(3, 2).$$

Self-Transitions in an Equation and an Update

If $\pi(3, 3) = \text{right}$, the fixed-policy equation is

$$V^\pi(3, 3) = 0.792 + 0.095V^\pi(3, 3) + 0.095V^\pi(3, 2).$$

Rearranging gives

$$0.905V^\pi(3, 3) = 0.792 + 0.095V^\pi(3, 2).$$

An Iterative Lookahead

Given an existing estimate V_k , compute

$$Q_{V_k}((3, 3), \text{right}) = 0.792 + 0.095V_k(3, 3) + 0.095V_k(3, 2).$$

A synchronous update reads the old value even when the next state equals the current state.

The Value Iteration Update

Start from a value table V_0 and repeatedly apply

$$V_{k+1}(s) = \max_a Q_{V_k}(s, a) = \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + \gamma V_k(s')].$$

- ▶ One sweep updates every nonterminal state.
- ▶ Every right-hand side uses the same old table V_k .
- ▶ Terminal values remain zero in this grid.

Synchronous Updates

Store the results in a separate table. State order then has no effect on the result of a sweep.

The Value Iteration Procedure

1. Initialize $V(s) = 0$ for every state.
2. Compute $W(s) = \max_a Q_V(s, a)$ for every nonterminal state, with $W(s) = 0$ at terminals.
3. Measure the largest change: $\delta = \max_s |W(s) - V(s)|$.
4. Replace the entire table: $V \leftarrow W$.
5. Repeat until a chosen value-error stopping condition holds.

The Value Iteration Procedure

1. Initialize $V(s) = 0$ for every state.
2. Compute $W(s) = \max_a Q_V(s, a)$ for every nonterminal state, with $W(s) = 0$ at terminals.
3. Measure the largest change: $\delta = \max_s |W(s) - V(s)|$.
4. Replace the entire table: $V \leftarrow W$.
5. Repeat until a chosen value-error stopping condition holds.

Outputs

Return the new value table V and a policy $\pi(s) \in \arg \max_a Q_V(s, a)$. We will introduce a stopping rule after tracing the updates.

The Initial Value Table

3	0.000	0.000	0.000	0
2	0.000	wall	0.000	0
1	0.000	0.000	0.000	0.000
	1	2	3	4

$$V_0(s) = 0$$

The initialization contains no estimated future return.

- ▶ The model still includes all transition rewards.
- ▶ The first sweep compares expected immediate rewards.
- ▶ Terminal labels in value grids show continuation values.

The First Action Comparison

At $(3, 3)$, use $V_0 = 0$:

Action	Expected immediate reward	Q_{V_0}
Up	$0.1(1) + 0.9(-0.04)$	0.064
Right	$0.8(1) + 0.2(-0.04)$	0.792
Down	$0.1(1) + 0.9(-0.04)$	0.064
Left	$1.0(-0.04)$	-0.040

The First Action Comparison

At $(3, 3)$, use $V_0 = 0$:

Action	Expected immediate reward	Q_{V_0}
Up	$0.1(1) + 0.9(-0.04)$	0.064
Right	$0.8(1) + 0.2(-0.04)$	0.792
Down	$0.1(1) + 0.9(-0.04)$	0.064
Left	$1.0(-0.04)$	-0.040

$$V_1(3, 3) = \max\{0.064, 0.792, 0.064, -0.040\} = 0.792.$$

The next-state expectation comes before the comparison across actions.

Values After One Sweep

3	-0.040	-0.040	0.792	0
2	-0.040	wall	-0.040	0
1	-0.040	-0.040	-0.040	-0.040
	1	2	3	4

V_1

The positive terminal can be reached in one transition from (3,3).

- ▶ Its best immediate expected reward is 0.792.
- ▶ Every other nonterminal state can choose an action with immediate reward -0.04 for every outcome.
- ▶ Values farther away have not yet incorporated the positive entry reward.

A Second Backup Beside the Terminal

For right at $(3, 3)$, use $V_1(3, 3) = 0.792$ and $V_1(3, 2) = -0.04$:

$$\begin{aligned}Q_{V_1}((3, 3), \text{right}) &= 0.8(1) + 0.1[-0.04 + 0.95(0.792)] \\&\quad + 0.1[-0.04 + 0.95(-0.04)] \\&= 0.86344.\end{aligned}$$

The Effect of a Blocked Move

The upward slip remains at $(3, 3)$. The robot pays -0.04 and still has an opportunity to reach the terminal later.

Right remains the maximizing action, so $V_2(3, 3) = 0.86344$.

Checkpoint: The Neighboring Cell

At $(2, 3)$, consider action right, using the first-sweep values:

$$V_1(3, 3) = 0.792, \quad V_1(2, 3) = -0.04.$$

1. Where do the intended move and the two slips lead?
2. What probability belongs to the self-transition?
3. Calculate $Q_{V_1}((2, 3), \text{right})$.

Neighboring-Cell Backup: Solution

The robot reaches (3, 3) with probability 0.8 and stays at (2, 3) with probability 0.2:

$$\begin{aligned}Q_{V_1}((2, 3), \text{right}) &= 0.8[-0.04 + 0.95(0.792)] + 0.2[-0.04 + 0.95(-0.04)] \\&= 0.55432.\end{aligned}$$

Neighboring-Cell Backup: Solution

The robot reaches $(3, 3)$ with probability 0.8 and stays at $(2, 3)$ with probability 0.2:

$$\begin{aligned} Q_{V_1}((2, 3), \text{right}) &= 0.8[-0.04 + 0.95(0.792)] + 0.2[-0.04 + 0.95(-0.04)] \\ &= 0.55432. \end{aligned}$$

A Comparison Still Matters

Up and down each give 0.00104; left gives -0.078 . Right is best, so $V_2(2, 3) = 0.55432$.

Use $V_1(3, 3)$ in this sweep, even if $V_2(3, 3)$ has already been computed.

Values After Two Sweeps

3	-0.078	0.554	0.863	0
2	-0.078	wall	0.462	0
1	-0.078	-0.078	-0.078	-0.078
	1	2	3	4

V_2

The positive reward now influences (2, 3) and (3, 2) through their possible transitions to (3, 3).

- ▶ (3, 2) also faces a risk of entering the negative terminal.
- ▶ Distant states still primarily reflect movement costs.
- ▶ Displayed values are rounded to three decimals.

Values After Three Sweeps

3	0.366	0.722	0.918	0
2	-0.114	wall	0.564	0
1	-0.114	-0.114	0.296	-0.114
	1	2	3	4

V_3

Each additional sweep carries continuation values across one more transition.

- ▶ Values now incorporate longer possible routes.
- ▶ The result averages successful moves, slips, and blocked moves.
- ▶ A single intended path does not determine a state's value.

Values After Five Sweeps

3	0.635	0.820	0.939	0
2	0.406	wall	0.631	0
1	0.126	0.270	0.451	0.150
	1	2	3	4

V_5

The start state can incorporate the positive reward along a five-transition route.

- ▶ Later sweeps also account for recovery after slips.
- ▶ Finite intermediate tables can differ substantially from the infinite-horizon solution.

The Finite-Horizon Interpretation

With $V_0 = 0$, $V_k(s)$ is the optimal expected discounted return over the next k transitions, stopping earlier only at a terminal state, with zero value afterward.

$$V_{k+1}(s) = \max_a \sum_{s'} T(s' \mid s, a) [R(s, a, s') + \gamma V_k(s')].$$

- ▶ The first decision uses V_k for the remaining k transitions.
- ▶ A finite-horizon policy may change as the remaining horizon decreases.
- ▶ Repeatedly executing one policy greedy with respect to V_k has its own value, V^{π_k} .

Why Value Iteration Converges

- ▶ Each update averages possible outcomes and discounts future values.
- ▶ After a full sweep, the largest absolute error relative to V^* is at most γ times its previous size.
- ▶ For $0 \leq \gamma < 1$, repeated sweeps approach V^* from any finite starting table.

Example: $\gamma = 0.95$

If the largest error is 0.20, after one sweep it is at most

$$0.95(0.20) = 0.19.$$

A discount factor closer to 1 gives a weaker error-reduction guarantee.

A Practical Stopping Rule

To stop without knowing V^* , measure the largest change after a synchronous sweep:

$$\delta_{k+1} = \max_s |V_{k+1}(s) - V_k(s)|.$$

Choose a maximum acceptable value error ϵ . For $0 < \gamma < 1$, stop when

$$\delta_{k+1} \leq \epsilon \frac{1 - \gamma}{\gamma}.$$

Accuracy of the Returned Table

- ▶ Return the newly computed table V_{k+1} .
- ▶ Every state's value is then within ϵ of its optimal value.
- ▶ At $\gamma = 0$, one sweep is exact.

A Numerical Stopping Threshold

For the grid, choose a maximum value error of $\epsilon = 0.001$:

$$\gamma = 0.95, \quad \delta_{k+1} \leq 0.001 \frac{0.05}{0.95} \approx 0.00005263.$$

- ▶ Use the largest change across all states.
- ▶ Compare full-precision values rather than the rounded grid labels.
- ▶ Return the newly computed table.

Values and Decisions

- ▶ This criterion bounds value error.
- ▶ An unchanged greedy policy across a few sweeps alone does not certify optimality.

Stopping on the Grid

For $\epsilon = 0.001$, the threshold is approximately 0.00005263:

New table	Largest change	Threshold satisfied?
V_{19}	0.00006160	No
V_{20}	0.00002889	Yes

The largest change at V_{20} is below the threshold, so

$$\max_s |V_{20}(s) - V^*(s)| \leq 0.001.$$

The Converged Value Table

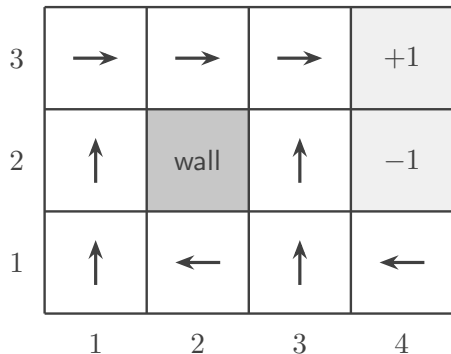
3	0.723	0.835	0.942	0
2	0.629	wall	0.641	0
1	0.531	0.449	0.517	0.284
	1	2	3	4

V^* , Rounded

The optimal expected return from the start is approximately 0.531.

- ▶ Values include movement costs, discounting, and random outcomes.
- ▶ Nearness to a terminal alone does not determine value.
- ▶ The terminal entries remain 0.

The Greedy Policy on the Grid



$$\pi^*(s) \in \arg \max_a Q_{V^*}(s, a).$$

- ▶ Arrows show intended actions.
- ▶ At (3, 2), choose up despite the possibility of slipping into the negative terminal.
- ▶ At (2, 1), choose left toward the safer side of the grid.

Policy Evaluation and Policy Improvement

Policy Evaluation

Hold the policy fixed and compute its expected returns:

$$V^{\pi}(s) = Q_{V^{\pi}}(s, \pi(s)).$$

Policy Improvement

Use those values to choose better actions:

$$\pi'(s) \in \arg \max_a Q_{V^{\pi}}(s, a).$$

Policy Evaluation and Policy Improvement

Policy Evaluation

Hold the policy fixed and compute its expected returns:

$$V^{\pi}(s) = Q_{V^{\pi}}(s, \pi(s)).$$

Policy Improvement

Use those values to choose better actions:

$$\pi'(s) \in \arg \max_a Q_{V^{\pi}}(s, a).$$

Policy Iteration

- ▶ Alternate evaluation and improvement until the policy no longer changes.
- ▶ Each evaluation uses the policy produced by the preceding improvement.

The Policy Iteration Procedure

1. Choose an initial deterministic policy π_0 .
2. Evaluate it: solve for V^{π_i} .
3. For every state, compare $Q_{V^{\pi_i}}(s, a)$ across actions.
4. Keep the current action if it is a maximizer; otherwise select a maximizing action to form π_{i+1} .
5. Stop if $\pi_{i+1} = \pi_i$; otherwise evaluate the new policy.

Two Different Objects

- ▶ The value table describes the current policy.
- ▶ The improvement step constructs the next policy.

The Initial Upward Policy

3	↑	↑	↑	+1
2	↑	wall	↑	-1
1	↑	↑	↑	↑
	1	2	3	4

$$\pi_0(s) = \text{up}$$

Start with the same intended action in every nonterminal state.

- ▶ At the top boundary, most outcomes stay in place.
- ▶ Sideways slips still occur.
- ▶ At (4, 1), the intended move enters the negative terminal.

Equations for the Upward Policy

At (1, 2), both perpendicular slips are blocked:

$$V^{\pi_0}(1, 2) = 0.8[-0.04 + 0.95V^{\pi_0}(1, 3)] + 0.2[-0.04 + 0.95V^{\pi_0}(1, 2)].$$

At (3, 3), an upward attempt stays in place:

$$V^{\pi_0}(3, 3) = 0.8[-0.04 + 0.95V^{\pi_0}(3, 3)] + 0.1[-0.04 + 0.95V^{\pi_0}(2, 3)] + 0.1(1).$$

Fixed Actions

There is no maximum in these equations. Every state follows up, including states reached after a slip.

Policy Evaluation as a Linear System

For a fixed policy, define

$$P_{\pi}(s, s') = T(s' \mid s, \pi(s)), \quad r_{\pi}(s) = \sum_{s'} P_{\pi}(s, s') R(s, \pi(s), s').$$

The policy equations become

$$V^{\pi} = r_{\pi} + \gamma P_{\pi} V^{\pi}, \quad (I - \gamma P_{\pi}) V^{\pi} = r_{\pi}.$$

- ▶ P_{π} is a transition matrix; r_{π} is an expected-reward vector.
- ▶ Solve simultaneously for all state values.
- ▶ Terminal values are known to be zero, so this grid needs nine nonterminal unknowns.

Values of the Upward Policy

3	-0.467	-0.292	0.151	0
2	-0.487	wall	-0.023	0
1	-0.500	-0.438	-0.186	-0.912
	1	2	3	4

V^{π_0} , Rounded

These values describe following up forever, until terminal entry ends the task.

- ▶ $V^{\pi_0}(1, 1) \approx -0.500$.
- ▶ $V^{\pi_0}(4, 1) \approx -0.912$.
- ▶ $V^{\pi_0}(3, 3) \approx 0.151$ because a rightward slip can reach the positive terminal.

Policy Improvement at One State

At $(3, 3)$, compare actions using the evaluated table V^{π_0} :

First action	$Q_{V^{\pi_0}}((3, 3), a)$
Up, the current action	0.151229
Right	0.804155
Down	0.018604
Left	-0.249485

Policy Improvement at One State

At $(3, 3)$, compare actions using the evaluated table V^{π_0} :

First action	$Q_{V^{\pi_0}}((3, 3), a)$
Up, the current action	0.151229
Right	0.804155
Down	0.018604
Left	-0.249485

The Improved Action

Choose right. Each entry above assumes that the robot follows π_0 after the first action. The value of the fully improved policy requires another evaluation.

The First Improved Policy

3	→	→	→	+1
2	↑	wall	↑	-1
1	→	→	↑	←
	1	2	3	4

π_1

Improve all states using the same evaluated table V^{π_0} .

- ▶ Top-row actions turn right.
- ▶ The bottom-right action turns left.
- ▶ At (1, 1) and (2, 1), the first improvement chooses right.

The Next Step

Evaluate π_1 before choosing the next set of improvements.

Values After the First Improvement

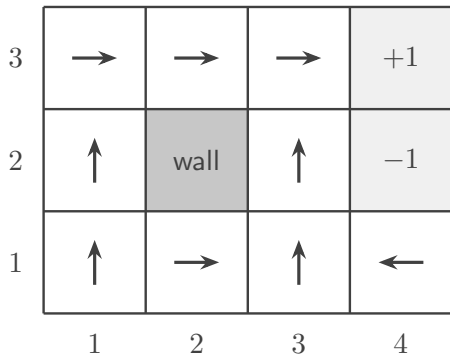
3	0.723	0.835	0.942	0
2	0.629	wall	0.641	0
1	0.386	0.434	0.515	0.283
	1	2	3	4

V^{π_1} , Rounded

The policy now has much higher expected return throughout the grid.

- ▶ At the start, value rises from about -0.500 to 0.386 .
- ▶ Improvements elsewhere affect the value of the start state.
- ▶ Further improvement remains possible on the bottom row.

The Second Improved Policy



π_2

Using V^{π_1} , only (1,1) changes: right becomes up.

- The improved top route now makes the left side attractive.
- Evaluating π_2 raises the start value to about 0.530.
- The action at (2,1) is still right.

The Stable Policy

3	→	→	→	+1
2	↑	wall	↑	-1
1	↑	←	↑	←
	1	2	3	4

π_3

Using V^{π_2} , only (2,1) changes: right becomes left.

- ▶ Evaluate π_3 and compare actions again.
- ▶ No action has a strictly better lookahead value.
- ▶ The result matches the policy obtained by value iteration.

The Policy Iteration Trace

Policy	How it was obtained	$\pi(1, 1)$	$\pi(2, 1)$	$V^\pi(1, 1)$
π_0	Up at every state	up	up	-0.500
π_1	First greedy improvement	right	right	0.386
π_2	Change at (1, 1)	up	right	0.530
π_3	Change at (2, 1)	up	left	0.531

Why Policy Improvement Works

1. Evaluate the current policy π exactly.
2. At each state, compare actions using V^π for the continuation.
3. Choose a maximizing action, keeping the current action whenever it is tied for best.

Improvement Guarantee

The resulting policy π' has at least as much expected return from every state:

$$V^{\pi'}(s) \geq V^\pi(s).$$

The current action was available in every comparison. Choosing the best actions together preserves this guarantee.

Policy Stability and Optimality

- ▶ Evaluate the current policy exactly, then compare all actions at every state.
- ▶ If no action has a higher expected return, every current action is already best for those values.
- ▶ The values therefore satisfy the Bellman optimality equation, and the policy is optimal.

Why Policy Iteration Stops

There are finitely many deterministic stationary policies. Each strict improvement raises the value of at least one state and lowers none. With exact evaluation and ties kept unchanged, an earlier policy cannot be revisited.

Ties and Numerical Evaluation

- ▶ Keep the current action whenever it is already a maximizer.
- ▶ Arbitrary switching between equally good actions can prevent an unchanged-policy stopping test from succeeding.
- ▶ Numerical solvers produce small errors, so implementations need a stated comparison tolerance.

Exact Theory and Computation

The guarantee assumes exact evaluation and strict improvements. For computed values, check the policy-evaluation equations and whether apparent improvements exceed numerical errors.

Value Iteration and Policy Iteration

	Value iteration	Policy iteration
Main object	A changing value table	A changing policy
Update	One optimality backup per state	Evaluate the policy, then improve it
Actions	Maximize during each sweep	Maximize during improvement
Stopping	Certified value error	Stable policy after exact evaluation
Shared target	Optimal expected discounted return in the known MDP	

Computational Work

Let n be the number of states, m the maximum number of actions, and d the maximum number of possible successors per state–action pair.

- ▶ One sparse value-iteration sweep costs $O(nmd)$.
- ▶ One policy-improvement pass has the same order of cost.
- ▶ A dense direct policy-evaluation solve costs $O(n^3)$; sparse systems can be cheaper.

A Useful Comparison

Count the work inside each iteration as well as the number of iterations. Neither algorithm is uniformly faster across all MDPs.

Main Takeaways

- ▶ Bellman equations express value as immediate reward plus discounted continuation value.
- ▶ Value iteration repeatedly applies optimality backups to a value table.
- ▶ Policy iteration alternates fixed-policy evaluation with greedy improvement.
- ▶ Reward timing, self-transitions, and update schedules determine the correct calculation.
- ▶ Convergence and stopping claims require their stated assumptions.

Thank You

Any questions?



References

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL <https://aima.cs.berkeley.edu/>.